

```

#<Pulkit Saxena>          <12.November.2023>
# <Assignment 5>
#<Description>:In this assignment we will learn about how to create lists and how
can we perform different operations like adding an item, deleting an item, moving
an item from one
#list to another etc. with the elements present in it.Furthermore, we have have
created functions such as savelist,loadlist etc. to save the list and load it on
the screen when asked
#by the user.In the end we have created PrintToDoList function which takes the
input from the user for whatever operations he/she would like to do with elements
present inside the list and then the functions have been assigned some variables
through which we are able to get the output.

import sys
import pickle
def printTitleMaterial():
    """Prints the title material for the game, including the student's name, class,
    and section number."""
print("The Ultimate TODO List!")
print()
print("By: <Pulkit Saxena>")
print("[COM S 127 <J>]")
print()
def initList():
    """Create a Dictionary of Lists - this is the primary data structure for the
    script.

    :return Dictionary of Lists: A dictionary whose keys contain the various
    categories the user can access. The values are lists the user can modify.
    """
    todoList = {
        "backlog": [],
        "todo": [],
        "in_progress": [],
        "in_review": [],
        "done": []
    }

    return todoList

def checkIfListEmpty(todoList):
    """This function checks if there are any entries in the Dictionary of Lists
    data structure.

    :param Dictionary of Lists todoList: A dictionary whose keys contain the
    various categories the user can access. The values are lists the user can modify.
    :return Boolean: If there is at least one item in the data structure, return
    False - it is not empty. Otherwise return True.
    """
    if (len(todoList["backlog"]) > 0 or
        len(todoList["todo"]) > 0 or
        len(todoList["in_progress"]) > 0 or
        len(todoList["in_review"]) > 0 or
        len(todoList["done"]) > 0):
        return False
    return True

def saveList(todoList):
    """Allows the user to save their data to a binary file.

```

```

:param Dictionary of Lists todoList: A dictionary whose keys contain the
various categories the user can access. The values are lists the user can modify.
"""
    try:
        listName = input("Enter List Name (Exclude .lst Extension): ")
        with open("./" + listName + ".lst", "wb") as pickle_file:
            pickle.dump(todoList, pickle_file)
    except:
        print("ERROR (saveList): ./{0}.lst is not a valid file
name!".format(listName))
        sys.exit()

def loadList():
    """Allows the user to load their data from a binary file.

    :return Dictionary of Lists: A dictionary whose keys contain the various
categories the user can access. The values are lists the user can modify.
"""
    try:
        listName = input("Enter List Name (Exclude .lst Extension): ")
        with open("./" + listName + ".lst", "rb") as pickle_file:
            todoList = pickle.load(pickle_file)
    except:
        print("ERROR (loadList): ./{0}.lst was not found!".format(listName))
        sys.exit()

    return todoList

def checkItem(item, todoList):
    """This function iterates through all the keys in the dictionary, and checks
each list to see if a key is present.

    :param String item: The String to search for in each list.
    :param Dictionary of Lists todoList: A dictionary whose keys contain the
various categories the user can access. The values are lists the user can modify.
    :return Boolean, String, Integer: This function returns True/ False depending
on whether the item was found, the String of the keyName, and the index in the list
where the item was found.
"""
    itemFound = False
    keyName = ""
    index = -1

    for key, value in todoList.items():
        if item in value:
            itemFound = True
            keyName = key
            index = value.index(item)
            break

    return itemFound, keyName, index

def addItem(item, toList, todoList):
    """This function allows the user to add an item to a specific list in the
todoList data structure.

    :param String item: The String to search for in each list.
    :param String toList: The key in the dictionary whose list to add the item to.

```

```

        :param Dictionary of Lists todoList: A dictionary whose keys contain the
various categories the user can access. The values are lists the user can modify.
        :return Dictionary of Lists: A dictionary whose keys contain the various
categories the user can access. The values are lists the user can modify.
    """

```

```

    if checkItem(item, todoList)[0]:
        print(f"Error: Item '{item}' already exists in list '{toList}'")
    else:
        todoList[toList].append(item)
    return todoList

```

```

def deleteItem(item, todoList):
    """This function allows the user to delete an item in the todoList data
structure.

```

```

        :param String item: The String to search for in each list.
        :param Dictionary of Lists todoList: A dictionary whose keys contain the
various categories the user can access. The values are lists the user can modify.
        :return Boolean, Dictionary of Lists: This function returns True/ False
depending on whether the item was found, and the modified Dictionary of Lists data
structure.
    """

```

```

    itemFound, keyName, index = checkItem(item, todoList)

```

```

    if itemFound:
        todoList[keyName].remove(item)
        print(f"Item '{item}' deleted.")

```

```

    else:
        print(f"Error: Item '{item}' does not exist in any list.")

```

```

    return itemFound, todoList

```

```

def moveItem(item, toList, todoList):
    """This function allows the user to move an item from one List in the
Dictionary of Lists to another.

```

```

        :param String item: The String to search for in each list.
        :param String toList: The key in the dictionary whose list to add the item to.
        :param Dictionary of Lists todoList: A dictionary whose keys contain the
various categories the user can access. The values are lists the user can modify.
        :return Dictionary of Lists: This function returns the modified Dictionary of
Lists data structure.
    """

```

```

    itemFound, todoList = deleteItem(item, todoList)

```

```

    if itemFound:

```

```

        todoList = addItem(item, toList, todoList)

    return todoList

def printTODOList(todoList):
    """This function prints out the contents of the Dictionary of Lists data
    structure.

    :param Dictionary of Lists todoList: A dictionary whose keys contain the
    various categories the user can access. The values are lists the user can modify.
    :return None: After printing out the contents of the Dictionary of Lists data
    structure, we are explicitly returning 'None.'
    """

    for key, value in todoList.items():
        print(f"{key}: {value}")

    return None

def runApplication(todoList):
    """This function provides the primary functionality for the application. It
    allows the user to add items to the data structure, move items,
    delete items, save the data structure to a binary file, and return to the main
    menu.

    :param Dictionary of Lists todoList: A dictionary whose keys contain the
    various categories the user can access. The values are lists the user can modify.
    :return Dictionary of Lists: This function returns the modified Dictionary of
    Lists data structure.
    """
    while True:
        print("-----")
        choice = input("APPLICATION MENU: [a]dd to backlog, [m]ove item, [d]elete
        item, [s]ave list, or [q]uit to main menu?: ")
        print()

        if choice == "a":

            item = input("Enter an item: ")
            addItem(item, "backlog", todoList)
            printTODOList(todoList)

        elif choice == "m":

            if not checkIfListEmpty(todoList):
                item = input("Enter an item to move: ")
                itemFound, keyName, index = checkItem(item, todoList)

                while not itemFound:
                    print(f"Error: Item '{item}' does not exist.")
                    item = input("Enter a different item: ")
                    itemFound, keyName, index = checkItem(item, todoList)

                toList = input("Enter the list you want to move the item to: ")

                while toList not in todoList:

```

```

        print(f"Error: List '{toList}' does not exist.")
        toList = input("Enter a different list: ")

        todoList = moveItem(item, toList, todoList)
        printTODOList(todoList)
    else:
        print("Error: No items to move!")

    pass
elif choice == "d":
    if not checkIfListEmpty(todoList):
        item = input("Enter an item to delete: ")
        _, todoList = deleteItem(item, todoList)
        printTODOList(todoList)
    else:
        print("Error: No items to delete!")

    pass
elif choice == "s":
    saveList(todoList)
    print("Saving List...")
    print()
    printTODOList(todoList)
elif choice == "q":
    print("Returning to MAIN MENU...")
    print()
    break
else:
    print("ERROR: Please enter [a], [m], [d], [s], or [q].")
    print()

return todoList

def main():
    """This is the main() function for the program. It contains all of the initial
    choices the user can make. These choices include:
    - Starting a new Dictionary of Lists.
    - Loading a previously saved Dictionary of Lists.
    - Quitting the script altogether.
    """
    taskOver = False

    while taskOver == False:
        print("-----")
        choice = input("MAIN MENU: [n]ew list, [l]oad list, or [q]uit?: ")
        print()
        if choice == "n":
            todoList = initList()

            printTODOList(todoList)

            runApplication(todoList)
        elif choice == "l":
            todoList = loadList()

```

```
        printTODOList(todoList)

        runApplication(todoList)
    elif choice == "q":
        taskOver = True
        print("Goodbye!")
        print()
    else:
        print("Please enter [n], [l], or [q]...")
        print()

if __name__ == "__main__":
    main()
```